

Ch 5 — Criteria-Based Test Design Overview

Old vs New View of Testing

Old: Testing divided by stage (unit, module, integration, system) — each phase treated as completely different.
New: Focus on **structures and criteria** rather than stages. Test design is the same across phases; only the model and test inputs change.

Model-Driven Test Design Workflow

Software Artifact → Model/Structure → Test Requirements
→ Test Specifications → Input Values → Test Cases
→ Test Scripts → Test Results → Pass/Fail

Testing criteria define **what must be tested**. The model determines the structure we test against.

Four Structures for Modeling Software

#	Structure	What It Tests	Applied To
1	Input Space	Input partitions, boundary values, equivalence classes	Source, Specs, Design, Use cases
2	Graphs	Control flow graphs, state machines, UML. Targets: node/edge/path coverage	Source, Specs
3	Logic	Boolean decisions. Targets: decision, condition, MC/DC coverage	Source, FSMs, Specs, DNF
4	Syntax	Grammar rules. Used in compiler, parser, input grammar testing	Source, Models, Integ, Input

Core Definitions (Ch 5)

Test Requirement (TR): A specific element of a software artifact that a test case must satisfy.
Examples: a statement, a branch, a condition, a path.

Coverage Criterion: A rule or collection of rules that impose test requirements on a test set. Examples: statement cov, branch cov, path cov, condition cov.

Coverage: Did our tests cover everything required? Test set T satisfies coverage if: for every TR, there exists at least one test case that satisfies it.

Coverage Level: = (# TRs satisfied) / (total TRs). E.g. 4 covered out of 6 = 4/6 = 66.7%.

Infeasible TR: A requirement that cannot be tested (dead code, unreachable paths). Detecting infeasibility is often **undecidable**. Therefore 100% coverage is rarely achievable.

Subsumption: C1 subsumes C2 if every test set satisfying C1 also satisfies C2. Example: branch coverage subsumes statement coverage (if every branch is tested, every statement is executed).

Two Ways to Use Test Criteria

Method 1: Generator — Criterion → Generate Tests

Strong coverage, systematic. Con: hard without automation.

Method 2: Recognizer — Tests → Measure Coverage

Most common in industry. Tools: JaCoCo, Istanbul, gcov.

Recognizing coverage is **much easier** than generating tests.

Advantages of Criteria-Based Testing

- **Better efficiency:** fewer tests, higher fault detection
- **Less redundancy:** tests overlap less
- **Traceability:** you know why each test exists
- **Regression support:** coverage helps ensure updates don't break tested areas
- **Stopping rule:** you know when testing is sufficient
- **Automation friendly:** criteria integrate well with tools

Good Coverage Criterion Properties

1) Easy to compute TRs. 2) Efficient test generation. 3) Detects many faults. Note: subsumption does NOT perfectly measure fault detection.

Monkey testing (random inputs, no structure) = low coverage, inefficient. Automation + criteria = better.

Ch 6 — Input Space Partitioning (ISP)

Goal

Reduce an extremely large (practically infinite) input space into **smaller groups (partitions)** so we can test efficiently. Instead of testing every possible input, we test **representative values from each group**.

Input Domain

= all possible inputs to a program. E.g. `triang(side1, side2, side3)` where each side can be any integer — practically infinite space.

Partition Properties (MUST satisfy both)

1. Disjoint: Blocks cannot overlap. $b_i \cap b_j = \emptyset$ for all $i \neq j$. Bad example: b_1 =ascending, b_2 =descending, b_3 =arbitrary — a 1-element file fits all three, so NOT disjoint.

2. Complete: Blocks cover the entire input domain. $b_1 \cup b_2 \cup \dots \cup b_n = D$ (entire domain).

ISP — 5 Modeling Steps (How To Do It)

Step 1 — Identify Testable Functions

Method, class, program, or system. E.g. `triang(side1, side2, side3)`

Step 2 — Find Parameters

All inputs: method params, files, globals, user input, DBs.

Step 3 — Model the Input Domain

For each parameter define **characteristics** and split into **blocks** with representative values.

Characteristic: Relation of side1 to 0
Blocks: $b_1 = >0$, $b_2 = =0$, $b_3 = <0$
Values: $>0 \rightarrow 5$, $=0 \rightarrow 0$, $<0 \rightarrow -1$

Step 4 — Choose Combinations

Combine blocks to create test cases. Different **coverage criteria** determine which combos to pick. E.g. $side1 > 0$, $side2 = 0$, $side3 < 0$.

Step 5 — Select Actual Values

Convert abstract blocks into real input values. E.g. $>0 \rightarrow 2$, $=0 \rightarrow 0$, $<0 \rightarrow -1$, final test = (2, 0, -1).

Two ISP Modeling Approaches

1. Interface-Based

Uses **input parameters directly**. Easy, mechanical. E.g. `triang()`: chars = side1/side2/side3 relation to 0. Blocks: >1 , $=1$, $=0$, <0 . Total = $4 \times 4 \times 4 = 64$. Con: ignores param relationships.

2. Functionality-Based

Uses **program behavior**. E.g. char = triangle type. Blocks: scalene(4,5,6), isosceles(3,3,4), equilateral(3,3,3), invalid(3,4,8). Better tests. Con: harder to design.

findElement(List, Object) Example

Interface: list is null(T/F), list is empty(T/F). **Functionality:** # occurrences(0,1,>1), element first(T/F), element last(T/F).

ISP Coverage Criteria — Detailed

ACoC → {TWC, MBCC} → {PWC, BCC} → ECC

1. All Combinations Coverage (ACoC)

Test **every possible combination** of blocks. Formula: $\#tests = \prod |B_i|$. E.g. $4 \times 4 \times 4 = 64$. Pro: maximum coverage. Con: too expensive, grows exponentially.

2. Each Choice Coverage (ECC)

Each block must appear in **at least one** test case. Formula: $\#tests = \max(|B_i|)$. E.g. $\max(4,4,4)=4$. Just line up blocks diagonally: (A1,B1,C1), (A2,B2,C2), (A3,B3,C3), (A4,B4,C4). Pro: very cheap. Con: may miss interactions between parameters.

3. Pair-Wise Coverage (PWC)

Every pair of values from any 2 characteristics must appear in ≥ 1 test. Formula: $\geq$ product of 2 largest. Good cost/coverage balance. Widely used. Captures 2-parameter interactions.

4. T-Wise Coverage (TWC)

Generalization: combos of **t** values. $t=2$ is PWC, $t=Q$ is ACoC. Formula: $\geq$ product of **t** largest. Expensive.

5. Base Choice Coverage (BCC)

Choose **one base test** (the "normal" or most important case). Then vary **one parameter at a time**, holding all others at base values.

Base: A1 B1 C1
Vary C: A1 B1 C2, A1 B1 C3, A1 B1 C4
Vary B: A1 B2 C1, A1 B3 C1
Vary A: A2 B1 C1, A3 B1 C1, A4 B1 C1

Formula: $1 + \sum(|B_i| - 1)$. E.g. $1+3+3+3 = 10$. Uses domain knowledge to pick important base values.

6. Multiple Base Choice Coverage (MBCC)

Same as BCC but with **M base tests** (multiple normal cases). Used when multiple normal cases exist. Formula: $M + M \cdot \sum(|B_i| - M)$. For each base, vary one parameter at a time through all non-base values.

Constraints Between Characteristics

Some combos are impossible (e.g. $side < 0$ AND $triangle = scalene$). Two types: **cannot combine** and **must combine**.

Handling: ACoC/PWC → drop invalid combos. BCC → replace value with another block.

Example: `findElement` — (1-element list, found >1 time) is invalid. (all identical, found exactly once) is invalid.

Ch 7 — Graph Coverage Criteria

Why Graphs?

Graphs are the **most common structure** for modeling software behavior. Testing = covering parts of the graph. Sources: control flow graphs (code), design structure, FSMs/ statecharts, use cases.

Graph Definition

$G = (N, N_0, N_f, E)$ where N = set of nodes ($\neq \emptyset$), N_0 = initial nodes ($\neq \emptyset$), N_f = final nodes ($\neq \emptyset$), E = edges ($n_i \rightarrow n_j$).

Path Definitions

Path: Sequence of nodes $[n_1, n_2, \dots, n_M]$.

Length: Number of edges. $[1,4,8]$ has length 2.

Subpath: Contiguous subsequence of a path.

Test Path: Path from initial node to final node. Represents execution of a test case.

SESE Graph: Single Entry, Single Exit. Exactly 1 initial + 1 final node. All test paths start and end at same nodes.

Visit vs Tour & Reachability

Visit: Path **visits** a node/edge if that node/edge is included in the path.

Tour: Path **tours** a subpath if it contains the entire subpath as a contiguous subsequence. Touring is **stronger** than visiting.

Example: Path [1,2,4,5,7]

Visits nodes: 1,2,4,5,7. Visits edges: (1,2),(2,4),(4,5),(5,7).

Tours subpaths: $[1,2,4]$, $[2,4,5]$, $[4,5,7]$, $[1,2,4,5]$, $[2,4,5,7]$, $[1,2,4,5,7]$.

Reachability

Syntactic reach: Path exists in graph structure.

Semantic reach: Path is actually executable with real inputs.

Not all syntactically reachable paths are executable! (Exam trick)

Tests → Test Paths

Deterministic: many-to-one — multiple tests can execute same path. **Non-deterministic:** many-to-many — same test can execute different paths.

Notation: $path(t)$ = path of test **t**. $path(T)$ = set of all test paths.

Structural Coverage Criteria (How To)

Process: 1) Model software as graph. 2) Define test requirements (TR) based on criterion. 3) Create test paths to satisfy them. Satisfaction: test set T satisfies criterion C iff for every TR, there exists a path in T that covers it. Two types: **Structural** (nodes/edges only) and **Data Flow** (needs variable def/use info).

Node Coverage (NC) — weakest

TR = all reachable nodes. Visit every node at least once.

Equivalent to statement coverage in code.

Example (nodes 1,2,3): $TR = \{1,2,3\}$. Test path = $[1,2,3]$ covers all.

Edge Coverage (EC)

TR = all reachable paths of length ≤ 1 (all nodes + all edges).

Visit every edge at least once. Equivalent to branch coverage.

EC subsumes NC because length ≤ 1 includes length 0 (nodes). Why " ≤ 1 " not just edges? A 1-node graph has no edges; without this, EC wouldn't include NC.

NC $\neq$ EC only when there's an edge AND another subpath between a pair of nodes (if-else). Example: nodes $\{1,2,3\}$, edges $\{(1,2),(1,3),(2,3)\}$. NC: path $[1,2,3]$ covers all nodes. EC needs $[1,2,3]$ AND $[1,3]$ to cover edge (1,3).

Edge-Pair Coverage (EPC)

TR = all reachable paths of length ≤ 2 . Tests must cover all sequences of 2 consecutive edges. EPC subsumes EC which subsumes NC.

Complete Path Coverage (CPC) — strongest, usually impossible

TR = ALL paths in graph. Impossible if loops exist (infinite paths).

Specified Path Coverage (SPC)

TR = user-defined set of paths. Subjective, depends on tester's judgment.

Simple Paths & Prime Paths

Simple Path Definition

A path where **no node repeats**, except possibly first = last (so loops $[n, \dots, n]$ can be simple). No internal loops allowed.

Prime Path Definition

A simple path that is **NOT a proper subpath of any other simple path**. Intuition: "maximal simple path" — can't be extended without breaking simplicity.

Example (4-node graph: 1→2, 1→3, 2→4, 3→4, 4→1)

Prime paths (8): $[2,4,1,2]$, $[2,4,1,3]$, $[1,3,4,1]$, $[1,2,4,1]$, $[3,4,1,2]$, $[4,1,3,4]$, $[4,1,2,4]$, $[3,4,1,3]$

All are length-3 simple paths that can't be extended. Shorter paths like $[1,2,4]$ are subpaths of $[1,2,4,1]$, so not prime.

Prime Path Coverage (PPC)

TR = all prime paths in the graph.

- **Finite** — always finite number of prime paths, even with loops
- **Handles loops**: covers executing loops 0, 1, and multiple times
- PPC subsumes NC and EC
- PPC does **NOT** subsume EPC (exam point!)

Why PPC does NOT subsume EPC

Self-loop: node n → n. EPC requires [n,n,m] and [m,n,n]. These repeat node n → NOT simple → NOT prime paths. PPC won't generate them.

Loop Prime Paths Example (nodes 1-7, loop 5→6→5)

```
[1,2,3,4,7] [1,2,3,5,7] [1,2,3,5,6] (Loop 0)
[1,3,4,7] [1,3,5,7] [1,3,5,6] (skip 2)
[6,5,7] (exit) [6,5,6] [5,6,5] (first=last)
```

Graph Coverage Hierarchy & Touring

CPC (impossible w/ loops) > PPC > EC > NC
CPC > EPC > EC > NC

PPC and EPC are incomparable due to self-loops.

Criterion	TR =	Key Fact
NC	All reachable nodes	= statement coverage. Weakest.
EC	Paths length ≤ 1	= branch coverage. Subsumes NC.
EPC	Paths length ≤ 2	All pairs of consecutive edges.
PPC	All prime paths	Handles loops. Finite. Practical best.
CPC	ALL paths	Impossible with loops (∞ paths).
SPC	User-defined paths	Subjective. Depends on tester.

Three Touring Types

Type	Rule	Basis
Direct Tour	Exact subpath match, no detours	—
Sidetrips	Extra edges OK, return to same node	Edge-based
Detours	Extra nodes OK, must rejoin later	Node-based

Best effort: cover TRs with direct tours first, then sidetrips for remaining. Infeasible TRs: dead code, contradictions (x>0 AND x<0). Feasibility is **undecidable**.

How To: Find Graph Coverage Test Paths

For NC: List all nodes, find path(s) visiting all

Graph: 1→2, 1→3, 2→3
TR = {1, 2, 3}
Test path: [1,2,3] visits all

For EC: List all edges, find paths covering all

TR = {(1,2), (1,3), (2,3)}
[1,2,3] covers (1,2),(2,3). [1,3] covers (1,3).
Need 2 test paths.

For EPC: List all length-2 subpaths

For each node n, enumerate all (in-edge, out-edge) pairs. E.g. node 4 with in:(2,4),(3,4) out:(4,5),(4,6) → TRs: [2,4,5], [2,4,6], [3,4,5], [3,4,6].

For PPC: Find all prime paths

1) List all simple paths. 2) Remove subpaths of longer ones. 3) Remaining = prime paths. 4) Find test paths touring each.

Double-Diamond SESE

1→{2,3}, {2,3}→4, 4→{5,6}, {5,6}→7
4 test paths: [1,2,4,5,7] [1,2,4,6,7]
[1,3,4,5,7] [1,3,4,6,7]

ISP Formulas Quick Reference

Criterion	Formula	Example: A(4) B(3) C(5) D(2)
ACoC	$\prod B_i $	$4 \times 3 \times 5 \times 2 = 120$
ECC	$\max(B_i)$	$\max(4,3,5,2) = 5$
BCC	$1 + \sum(B_i - 1)$	$1 + 3 + 2 + 4 + 1 = 11$
MBCC (M)	$M + M \cdot \sum(B_i - M)$	$M=2: 2 + 4 + 2 + 6 + 0 = 14$
PWC	$\geq$ product of 2 largest	$5 \times 4 = 20$
TWC (t)	$\geq$ product of t largest	$t=3: 5 \times 4 \times 3 = 60$

How to do BCC & MBCC

BCC

1) Pick one base value per char (most normal). 2) Form base test. 3) For each char, change only that one to each non-base block, hold rest at base. Count = $1 + \sum(|B_i| - 1)$.

MBCC

1) Pick M base values per char. 2) Form M base tests. 3) For each base, vary one char at a time through non-base values (|B_i|-M per char). Total = $M + M \cdot \sum(|B_i| - M)$.

BCC Worked Example (11 tests)

A(4) B(3) C(5) D(2). Base: a1, b1, c1, d1. Formula: $1 + (4-1) + (3-1) + (5-1) + (2-1) = 11$

#	A	B	C	D	Changed
T1	a1	b1	c1	d1	Base
T2	a2	b1	c1	d1	A
T3	a3	b1	c1	d1	A
T4	a4	b1	c1	d1	A
T5	a1	b2	c1	d1	B
T6	a1	b3	c1	d1	B
T7	a1	b1	c2	d1	C
T8	a1	b1	c3	d1	C
T9	a1	b1	c4	d1	C
T10	a1	b1	c5	d1	C
T11	a1	b1	c1	d2	D

ECC Worked Example (5 tests)

max(4,3,5,2) = 5. Diagonal lineup, wrap shorter chars:

#	A	B	C	D
T1	a1	b1	c1	d1
T2	a2	b2	c2	d2
T3	a3	b3	c3	d1
T4	a4	b1	c4	d2
T5	a1	b2	c5	d1

Every block appears ≥1 time. Wrap shorter chars (B at T4, D at T3). Cheapest but may miss interactions.

MBCC Worked Example (M=2, 14 tests)

Bases: a1,a2|b1,b2|c1,c2|d1,d2. Non-base: A{a3,a4} B{b3} C{c3,c4,c5} D{none}

#	A	B	C	D	Note
T1	a1	b1	c1	d1	Base 1
T2	a2	b2	c2	d2	Base 2
T3-4	a3,a4	b1	c1	d1	Vary A from B1
T5	a1	b3	c1	d1	Vary B from B1
T6-8	a1	b1	c3,c4,c5	d1	Vary C from B1
T9-10	a3,a4	b2	c2	d2	Vary A from B2
T11	a2	b3	c2	d2	Vary B from B2
T12-14	a2	b2	c3,c4,c5	d2	Vary C from B2

PWC & TWC Verification

Pair-Wise: 20 tests (product of 2 largest = 5×4)

Every pair from any 2 chars in ≥1 test. Cover A{a3,a4} bottleneck first, fill B/D.

(A,B):12	(A,C):20	bottleneck	(A,D):8
(B,C):15	(B,D):6		(C,D):10

Total: 71 pairs across 20 tests.

T-Wise t=3: 60 tests (5×4×3)

Enumerate 60 (A,B,C) triples. Assign D by parity. Verify C(4,3)=4 triplet sets.

(A,B,C):60	bottleneck	(A,B,D):24	(A,C,D):40	(B,C,D):30
------------	-------------------	------------	------------	------------

ISP & Graph Hierarchies Summary

ISP Subsumption

ACoC > TWC > PWC > ECC | ACoC > MBCC > BCC > ECC
Higher = more tests = more coverage. t=Q → TWC=ACoC.

Graph Subsumption

CPC > PPC > EC > NC | CPC > EPC > EC > NC

PPC does NOT subsume EPC. CPC impossible with loops. Branch cov subsumes statement cov.

Full Graph Coverage Walkthrough

Graph: N={0,1,2,3,4,5}, edges: 0→1, 0→4, 1→2, 1→3, 2→3, 3→5, 4→5. N₀= {0}, N_f= {5}.

NC: TR = {0,1,2,3,4,5}

Paths: [0,1,2,3,5] covers {0,1,2,3,5}. Missing: 4. Add [0,4,5]. **2 test paths.**

EC: TR = {(0,1),(0,4),(1,2),(1,3),(2,3),(3,5),(4,5)}

[0,1,2,3,5] covers (0,1),(1,2),(2,3),(3,5). [0,4,5] covers (0,4),(4,5). Missing: (1,3). Add [0,1,3,5]. **3 test paths.**

EPC: TR = all length-2 subpaths

```
[0,1,2] [0,1,3] [0,4,5] [1,2,3] [1,3,5] [2,3,5]
Paths: [0,1,2,3,5] tours [0,1,2],[1,2,3],[2,3,5]
       [0,1,3,5] tours [0,1,3],[1,3,5]
       [0,4,5] tours [0,4,5]
3 test paths (same as EC here)
```

PPC: find prime paths

Simple paths: [0,1,2,3,5] [0,1,3,5] [0,4,5]
[4,5] [2,3,5] etc. Primes (can't extend):
[0,1,2,3,5] [0,1,3,5] [0,4,5]
All are already test paths! 3 paths.

How To: Code → CFG → Coverage (computeStats)

Nodes 1-11. Two loops: 3→4→5→3 (loop1) and 8→9→10→8 (loop2). N₀= {1}, N_f= {11}.

Edges(12): (1,2)(2,3)(3,4)(3,6)(4,5)(5,3)
(6,7)(7,8)(8,9)(8,11)(9,10)(10,8)

Step 3: Edge Coverage

One path can cover all 12 edges by taking each loop once:

[1,2,3,4,5,3,6,7,8,9,10,8,11]

Step 4: Edge-Pair Coverage (15 TRs)

TRs: (1,2,3)(2,3,4)(2,3,6)(3,4,5)(3,6,7)
(4,5,3)(5,3,4)(5,3,6)(6,7,8)(7,8,9)
(7,8,11)(8,9,10)(9,10,8)(10,8,9)(10,8,11)

Key: (5,3,4) & (5,3,6) need loop1 ≥2x. (10,8,9) & (10,8,11) need loop2 ≥2x.

[1,2,3,4,5,3,4,5,3,6,7,8,9,10,8,9,10,8,11]
[1,2,3,6,7,8,11] (covers 2,3,6 + 7,8,11)

Step 5: Prime Paths (12 TRs)

Round-trips: PP1(3,4,5,3) PP2(4,5,3,4)
PP3(5,3,4,5) PP4(8,9,10,8) PP5(9,10,8,9)
PP6(10,8,9,10)
Maximal: PP7(1,2,3,4,5) PP8(1,2,3,6,7,8,11)
PP9(1,2,3,6,7,8,9,10) PP10(4,5,3,6,7,8,11)
PP11(4,5,3,6,7,8,9,10) PP12(9,10,8,11)

4 test paths satisfy all 12 PPs:

[1,2,3,4,5,3,4,5,3,6,7,8,11] PP1,2,3,7,10
[1,2,3,6,7,8,9,10,8,9,10,8,11] PP4,5,6,9,12
[1,2,3,6,7,8,11] PP8
[1,2,3,4,5,3,6,7,8,9,10,8,11] PP11

CFG Building Rules

- Sequential statements (no branches) = **one node**
- if/loop condition = **decision node** with 2 out-edges (T/F)
- for loop: condition → body → back-edge to condition. False exits.
- Method entry = N₀. Return/end = N_f.

```
if(c){A;}else{B};C    for(init;c;inc){body}
[c]                  [init]+[c]←++
T/ \F                T/ \F      |
[A] [B]              [body]+[inc]-+
                    \ /
                    [next]
```

Common Exam Traps

- PPC does NOT subsume EPC (self-loop creates non-simple edge-pair paths)
- CPC is impossible when loops exist (infinite paths)
- EC uses "length ≤ 1" not just "edges" — needed so EC subsumes NC for 1-node graphs
- Subsumption does NOT perfectly measure fault detection
- 100% coverage rarely achievable (infeasible TRs, undecidable)
- Syntactically reachable ≠ semantically reachable (executable)
- Partition must be both disjoint AND complete
- ECC formula = max(|B_i|), NOT sum
- BCC formula = $1 + \sum(|B_i| - 1)$, NOT product
- PWC lower bound = product of 2 LARGEST, not 2 smallest
- MBCC non-base values = |B_i|-M (subtract M, not 1)
- Simple path allows first=last (loops are simple), but no other repeats
- Prime path = maximal simple path (can't be extended)
- Deterministic software: many tests → one path. Non-det: many-to-many.